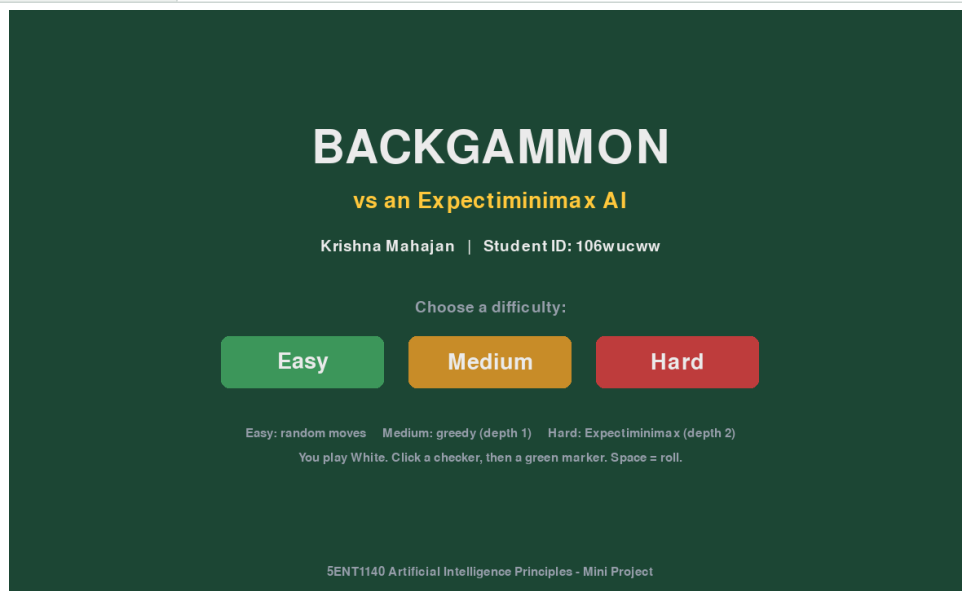


Backgammon AI

An Expectiminimax opponent for a game of chance

Name	Krishna Mahajan
Student ID	106wucww
Module	5ENT1140 Artificial Intelligence Principles
Institution	University of Hertfordshire (delivered at PSB Academy, Singapore)
Submission	Mini Project: published game, presentation, report and code
Playable game	https://kris072008.itch.io/backgammon-ai



Title screen of the published game

1. Introduction

The aim of this project was to implement a two-player game that involves chance, in which the computer evaluates every possible consequence of each move and chooses the one that is closest to victory. I chose backgammon. It is a genuinely stochastic game: before every turn the dice decide which moves are even possible, so a player cannot plan against a single fixed future. This makes ordinary Minimax unsuitable and calls for Expectiminimax, which extends Minimax with chance nodes that average outcomes by their probability (Russell and Norvig, 2021). Backgammon is also the classic benchmark for this family of methods, most famously in Tesauro's TD-Gammon (1995).

The finished program implements the full rules: 15 checkers each on 24 points, doubles played four times, hitting single checkers (blots) onto the bar, compulsory re-entry from the bar, bearing off, and the rules that a player must use as many dice as possible and must use the larger die when only one can be played. Three computer opponents are provided: Easy (random legal moves), Medium (greedy, one move of look-ahead) and Hard (depth-2 Expectiminimax). The game has a graphical interface written with Pygame and is published online, playable in any browser, at kris072008.itch.io/backgammon-ai. The report also traces one real decision by the Hard AI step by step, and compares the three methods over more than 550 AI-versus-AI games.

2. Design

2.1 System architecture

The code is split into small modules so that the game rules, the AI and the interface can be developed and tested independently. The board is stored as a list of 24 integers: a positive number is that many White checkers, a negative number is Black checkers. Two small dictionaries hold the checkers on the bar and those borne off.

File	Responsibility
backgammon.py	Rules engine: board state, dice, legal-move generation, applying moves, win detection, terminal test mode.
ai.py	Evaluation function, Expectiminimax search, the three difficulty levels and a benchmark mode.
gui.py	Pygame interface: menu, board drawing, click-to-move, dice animation, AI move animation, game-over screen.
main.py	Entry point; runs the asynchronous game loop required for the browser version (pygbag).
experiments.py	AI-vs-AI tournaments; produces the results table and charts used in Section 3.5.

2.2 Interface design (wireframes)

Figure 1 shows the two screens. The title screen displays the student details required by the brief and lets the player choose a difficulty. On the game screen the board is drawn from White's point of view; legal sources are ringed in gold and, once a checker is selected, its legal destinations are shown as green markers, so the player can never make an illegal move. The side panel holds the dice, the Roll, Undo and Hint buttons, pip counts, an AI analysis box and a message log.

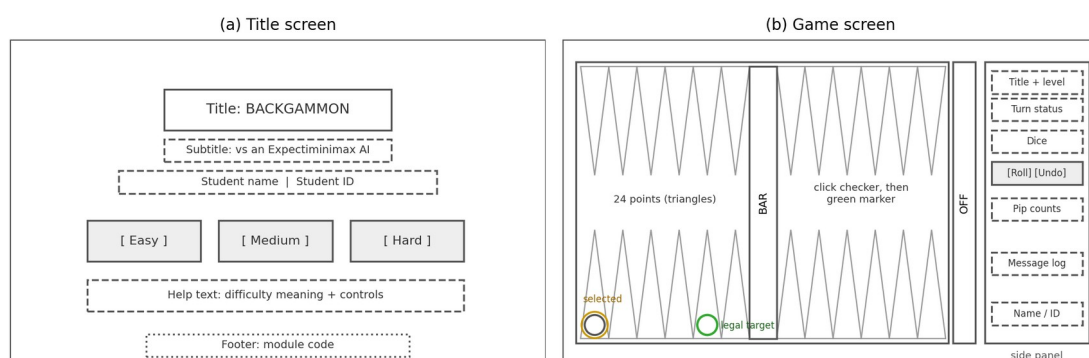


Figure 1. Wireframes of (a) the title screen and (b) the game screen.

2.3 AI design: Expectiminimax

The search tree has three kinds of node (Figure 2). At a MAX node the AI chooses the turn with the highest value; at a MIN node the opponent is assumed to choose the turn that is worst for the AI; and at a CHANCE node the value is the probability-weighted average over the 21 distinct dice rolls (doubles have probability 1/36, other rolls 2/36):

$$\text{value}(\text{chance node}) = \sum_r P(r) \times \text{value}(\text{best decision after roll } r)$$



Figure 3. The GUI during play. Left: a Hint, with the Hard AI's suggested move drawn as numbered arrows and its top three moves in the side panel; the bar at the top shows the estimated win chance and the badge rates the previous move. Right: after an AI turn, its last move ringed in blue and its analysis in the panel.

3.3 Sample input and output: terminal mode

The terminal test mode shows the engine's input and output most clearly. Below, White (the human) rolls 3-1 at the start; the program lists all 16 legal turns, the player enters turn 5 (the standard opening 8/5 6/5), and the Hard AI replies to a 6-4 by making a new point in its home board rather than leaving blots.

```

13 14 15 16 17 18 19 20 21 22 23 24
W5 . . . B3 . B5 . . . W2
B5 . . . W3 . W5 . . . B2
12 11 10 9 8 7 6 5 4 3 2 1

White rolled [3, 1]
0: 6/3 3/2 1: 6/3 6/5 2: 6/3 8/7
3: 6/3 24/23 4: 8/5 5/4 5: 8/5 6/5
... (16 legal turns in total)
Choose a turn number: 5
Black rolled [6, 4]
Computer plays: 17/23 19/23

13 14 15 16 17 18 19 20 21 22 23 24
W5 . . . B2 . B4 . . . B2 W2
B5 . . . W2 . W4 W2 . . . B2
12 11 10 9 8 7 6 5 4 3 2 1

```

3.4 Inside one decision: the opening 5-3

Input: the standard starting position; the Hard AI plays White and rolls 5-3. **Processing:** the AI generates its legal turns, keeps the six best by immediate score, and for each one searches all 21 opponent rolls and the opponent's best reply. Table 1 shows the real values returned by `hard_ai()`.

Candidate turn	Immediate score	Expected value (21 rolls)
8/3 6/3 (chosen)	24	15.83
13/8 13/10	15	4.19
13/8 6/3	15	-1.83
13/8 8/5	15	-3.17
13/8 24/21	9	-7.72
8/3 13/10	11	-8.47

Table 1. The six candidate turns searched for an opening 5-3.

Opponent roll (P)	8/3 6/3	13/8 13/10	13/8 24/21
1-2 (2/36)	21	12	-2
3-3 (1/36)	12	-16	-16
6-6 (1/36)	0	-9	-20

Table 2. Value after the opponent's best reply (MIN node) for three of the 21 rolls.

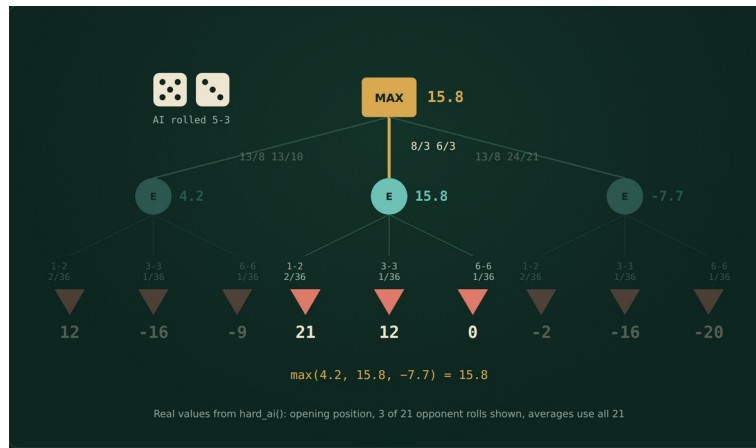


Figure 4. Frame from the explainer animation: MIN values feed the chance averages, and MAX takes the best.

Output: the AI plays 8/3 6/3, making its 3-point. It leaves no blots, so even the opponent's best roll (6-6) only lowers the value to 0, while 13/8 13/10 scores well on most rolls but is punished by 3-3 down to -16. The chance node judges each move by its average over every roll, not by one lucky or unlucky outcome. 8/3 6/3 is also the standard expert opening for 5-3, a good sign that the evaluation weights are sensible.

3.5 Experimental evaluation

To compare the methods, experiments.py plays silent AI-versus-AI games, swapping colours halfway through each match so neither side benefits from moving first. Hard vs Medium was run twice with different random seeds to check how much the result varies.

Experiment	Games	Win rate of first player	Gammons won	Thinking time
Medium vs Easy	100	100%	97	0.2 ms/move
Hard vs Easy	50	100%	47	60 ms/move
Hard vs Medium (run 1)	100	52%	21	54 ms/move
Hard vs Medium (run 2)	100	59%	-	50 ms/move
Hard vs Medium (combined)	200	55.5%	-	≈ 52 ms/move
Hard (beam 1 / 3 / 6 / 10) vs Medium	40 each	47.5 / 55 / 57.5 / 65%	-	rises with beam

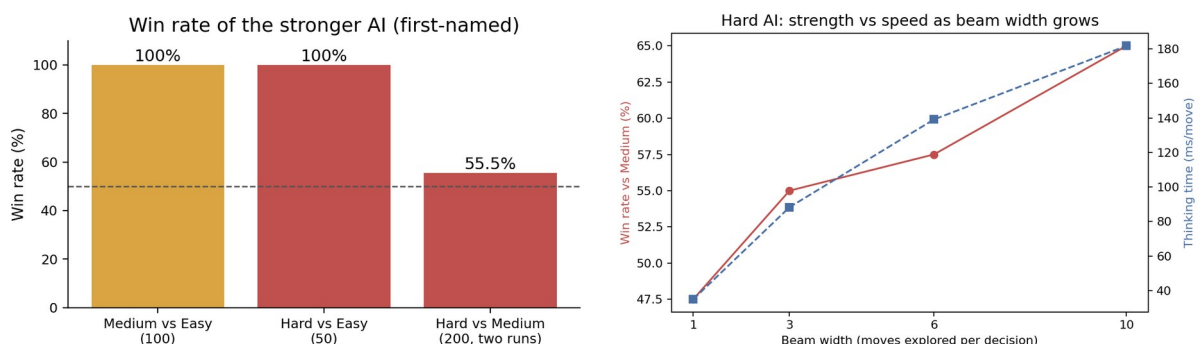


Figure 5. Left: tournament win rates. Right: effect of beam width on the Hard AI's strength and thinking time.

Any evaluation function is overwhelmingly better than random play: Medium won every game against Easy, almost all by gammon. Adding a full look-ahead over the opponent's dice gives a smaller but real gain. The two Hard vs Medium runs (52% and 59%) show how much luck there is in 100 games of backgammon; combined, Hard won 111 of 200 (55.5%). The gain grows as the search explores more candidate moves, from 47.5% with a beam of one to 65% with ten. This is a clear strength-versus-speed trade-off: every extra candidate costs another 21 dice rolls of opponent replies. The default beam of six keeps each move well under a second in the browser.

4. Conclusion

The project delivers a complete, playable backgammon game with three AI opponents, a clear graphical interface that shows the AI's reasoning, a calibrated win chance, hints and move grades, and an online version that runs in any browser. The Hard AI evaluates every legal turn, averages over every possible dice roll and assumes the best reply from the

opponent, meeting the brief's requirement to choose the move that is closest to victory. Traced through a real decision, it finds the expert opening for 5-3. The experiments confirmed that search and evaluation matter, and showed honestly that in a game dominated by dice the benefit of deeper search is smaller than in deterministic games.

Challenges. Enforcing the must-use-both-dice and larger-die rules required generating complete turns rather than single moves. Pygame 2.6 was incompatible with Python 3.14, which was solved by switching to pygame-ce. The web build first showed a blank screen because pygbag only detects libraries imported in main.py, and browsers do not support threads; both were fixed by importing Pygame in the entry file and running the search in the main loop. Later, the web build stalled on its loading screen because pygbag also tried to load matplotlib, which is only used by experiments.py; that file is now left out of the web build. Sound was silent in the browser because the browser chooses its own audio format; the sounds are now built from raw samples converted to whatever format the mixer uses.

Limitations and future work. The evaluation weights were set by hand; learning them through self-play, as TD-Gammon did, would likely strengthen the AI considerably. The beam of six can prune a good move whose immediate score is low. Alpha-beta style pruning for chance nodes (*-minimax), a deeper search, the doubling cube and a mobile-friendly layout are further improvements.

Use of AI tools. AI assistance (Claude) was used to support code development, debugging and the explainer animation; the design decisions, testing, experiments and analysis in this report are my own.

References

- Russell, S. and Norvig, P. (2021) *Artificial Intelligence: A Modern Approach*. 4th edn. Pearson (Ch. 5, Stochastic games).
- Tesauro, G. (1995) 'Temporal difference learning and TD-Gammon', *Communications of the ACM*, 38(3), pp. 58-68.
- Michie, D. (1966) 'Game-playing and game-learning automata', in Fox, L. (ed.) *Advances in Programming and Non-numerical Computation*. Pergamon.
- pygame-ce and pygbag documentation (2026). Available at: pyga.me and pygame-web.github.io.